

COMPTE RENDU D'ACTIVITÉ

Atelier de professionnalisation n°2

Développement de l'application MediaTek86

Mimoun Chaouchi

BTS SIO - Option SLAM

Session 2026

1. Présentation du contexte

1.1. L'entreprise InfoTech Services 86

Dans le cadre de cet atelier, j'ai été missionné en tant que technicien développeur junior au sein de l'ESN InfoTech Services 86 (ITS 86). Cette entreprise, créée en 2002, est une société de services numériques spécialisée dans le développement informatique, l'hébergement de sites web, l'infogérance, la gestion de parc informatique et l'ingénierie système et réseau.

ITS 86 est organisée autour de deux pôles : le pôle Développement et le pôle Systèmes et Réseau. L'entreprise compte 32 collaborateurs et accompagne ses clients TPE et PME dans des projets innovants.

1.2. Le client : MediaTek86

MediaTek86 est un réseau qui fédère les médiathèques du département de la Vienne. Sa mission principale est de mutualiser les prêts de livres, DVD et CD, et de développer la médiathèque numérique pour l'ensemble des médiathèques du département.

Le service administratif de MediaTek86 a confié à ITS 86 la réalisation d'une application de bureau permettant de gérer le personnel de chaque médiathèque, leur affectation à un service, ainsi que leurs absences.

1.3. La mission confiée

La mission qui m'a été confiée consiste à développer une application de bureau monoposte, destinée à être installée sur un poste du service administratif. L'application doit permettre au responsable du personnel :

- De se connecter de manière sécurisée à l'application
- De consulter la liste du personnel des médiathèques
- D'ajouter, modifier ou supprimer un personnel
- De consulter, ajouter, modifier ou supprimer les absences de chaque agent

2. Choix techniques

2.1. Technologies retenues

Conformément aux consignes de l'atelier, l'application a été développée avec les mêmes technologies que l'application Habilitations étudiée précédemment :

- **Langage** : C#
- **Framework** : .NET Framework 4.7.2
- **Interface graphique** : Windows Forms
- **IDE** : Visual Studio 2022 Entreprise
- **Base de données** : MySQL (via WampServer)
- **Versionning** : Git avec dépôt distant sur GitHub
- **Modélisation** : Looping pour le MCD
- **Documentation technique** : SandCastle Help File Builder

2.2. Architecture MVC

L'application a été structurée selon le modèle MVC (Modèle-Vue-Contrôleur), avec une organisation en 5 packages :

- **bddmanager** : contient la classe technique BddManager (singleton) qui gère la connexion à la base de données MySQL
- **dal** : (Data Access Layer) contient la classe Access qui regroupe toutes les requêtes SQL et expose des méthodes au contrôleur
- **modele** : contient les classes métier (Personnel, Service, Motif, Absence) correspondant aux tables de la base de données
- **controller** : contient les contrôleurs qui font le lien entre la vue et la DAL
- **vue** : contient les formulaires Windows Forms (FrmConnexion, FrmPersonnel, FrmAbsence, FrmPersonnelModif, FrmAbsenceModif)

2.3. Sécurité

La sécurité de l'application repose sur deux niveaux :

- **Un utilisateur MySQL dédié** : mediatek86_user, avec des droits limités à SELECT, INSERT, UPDATE et DELETE sur la base mediatek86. La connexion depuis l'application utilise cet utilisateur restreint plutôt que le compte root
- **Un hashage du mot de passe responsable** : le mot de passe saisi par le responsable lors de la connexion est hashé en SHA2-256 côté MySQL et comparé à la valeur stockée dans la table responsable. Le mot de passe en clair ne circule donc jamais dans la requête au-delà du paramètre sécurisé

3. Démarche de réalisation

3.1. Étape 1 - Mise en place de l'environnement et de la base de données

La première étape a consisté à installer et configurer l'environnement de développement : WampServer, Visual Studio 2022 Entreprise, et Looping.

À partir du MCD fourni et ouvert dans Looping, j'ai généré le script SQL correspondant au format MySQL. J'ai ensuite enrichi ce script pour y ajouter la création de la base mediatek86, la création de l'utilisateur dédié avec ses droits, la table responsable (non présente dans le MCD), et toutes les données de test.

La base finale contient 5 tables : responsable (1 entrée), service (3 entrées), motif (4 entrées), personnel (10 entrées) et absence (50 entrées).

3.2. Étape 2 - Structure MVC et interfaces visuelles

Après étude du dossier documentaire contenant le diagramme de cas d'utilisation, j'ai créé la solution Visual Studio en respectant la même structure de packages que l'application Habilitations. Le choix a été fait de regrouper certains cas d'utilisation dans un même formulaire pour faciliter les manipulations :

- FrmConnexion regroupe le cas Se connecter
- FrmPersonnel regroupe l'affichage, l'ajout, la modification et la suppression du personnel
- FrmPersonnelModif est le formulaire de saisie utilisé pour l'ajout et la modification
- FrmAbsence regroupe l'affichage, l'ajout, la modification et la suppression des absences

- FrmAbsenceModif est le formulaire de saisie utilisé pour l'ajout et la modification

Le dépôt distant GitHub a été créé et un projet Kanban a été mis en place avec les différentes issues correspondant aux tâches à réaliser.

3.3. Étape 3 - Modèle, BddManager et documentation

La classe BddManager (singleton) reprise de l'application Habilitations a été placée dans le package bddmanager. La classe Access du package dal a été initialisée avec la chaîne de connexion utilisant l'utilisateur dédié.

Les classes métier ont été créées dans le package modele, avec un constructeur, des propriétés et une redéfinition de ToString() pour faciliter l'affichage dans les ComboBox.

La documentation technique a été générée à l'aide de SandCastle Help File Builder à partir des commentaires XML normalisés. Le dossier de documentation a été zippé et intégré au dépôt.

3.4. Étape 4 - Codage des fonctionnalités

Chaque cas d'utilisation a été implémenté dans le respect du MVC :

- **Se connecter** : la méthode ControleAuthentification de la classe Access exécute une requête paramétrée avec SHA2(@pwd, 256) ; si l'authentification réussit, FrmPersonnel s'ouvre
- **Ajouter un personnel** : ouvre FrmPersonnelModif en mode ajout ; la validation vérifie que tous les champs sont remplis
- **Modifier un personnel** : ouvre FrmPersonnelModif avec les champs préremplis ; une confirmation est demandée avant validation
- **Supprimer un personnel** : demande une confirmation, puis supprime également les absences associées pour respecter la contrainte de clé étrangère
- **Afficher les absences** : ouvre FrmAbsence avec la liste triée par date de début décroissante
- **Ajouter/modifier une absence** : vérifie que la date de fin est postérieure à la date de début, et qu'aucune autre absence du même personnel ne chevauche le créneau saisi
- **Supprimer une absence** : demande confirmation avant suppression

3.5. Étape 5 - Vidéo de démonstration

Une vidéo d'environ 5 minutes a été enregistrée pour présenter toutes les fonctionnalités de l'application. Elle est commentée à l'oral et intégrée dans la page portfolio.

3.6. Étape 6 - Déploiement, Readme et portfolio

Un installateur MSI a été généré à l'aide de l'extension Microsoft Visual Studio Installer Projects. Il crée automatiquement les raccourcis dans le menu Démarrer et sur le bureau.

Le Readme du dépôt a été rédigé sur le modèle de celui de l'application Habilitations, incluant le contexte, l'architecture, le MCD, les interfaces, le diagramme de paquetages, l'installation, ainsi que le détail des commits.

Une page dédiée a été créée dans mon portfolio Wix, contenant la présentation du contexte, le lien vers le dépôt GitHub, le lien vers ce compte rendu et la vidéo de démonstration intégrée.

4. Difficultés rencontrées et solutions

4.1. Problèmes de namespace

Après ajout des classes NamespaceDoc pour la documentation, l'application ne se compilait plus car le namespace des formulaires différait entre le fichier .cs (MediaTek86) et le fichier .Designer.cs (MediaTek86.vue). La solution a été d'aligner tous les fichiers sur le namespace MediaTek86.vue, conforme à l'organisation en packages.

4.2. Événements de clic non branchés

Plusieurs boutons (Enregistrer, Annuler) ne réagissaient pas malgré la présence du code dans les méthodes. Le problème provenait du fait que les événements Click n'étaient pas correctement reliés aux méthodes dans le fichier Designer. La solution a été de brancher manuellement les événements via la fenêtre Propriétés > Événements de Visual Studio.

4.3. Verrouillage du fichier exécutable

Lors de plusieurs lancements rapprochés de l'application, le fichier MediaTek86.exe restait verrouillé en mémoire, empêchant la recompilation. La solution a été d'utiliser le gestionnaire des tâches Windows pour terminer les processus MediaTek86 résiduels.

4.4. Suppression d'un personnel ayant des absences

Tenter de supprimer directement un personnel ayant des absences déclenchait une erreur de contrainte de clé étrangère. La solution a été de supprimer d'abord les absences associées dans la méthode DeletePersonnel, avant de supprimer le personnel lui-même.

4.5. Détection de chevauchement des absences

Pour respecter le scénario alternatif 4c du cahier des charges ("Une absence est déjà programmée dans ce créneau"), une méthode AbsenceChevauche a été ajoutée dans la DAL. Elle utilise la condition SQL NOT (datefin < @datedebut OR datedebut > @datefin) pour détecter tout chevauchement, et exclut l'absence en cours de modification grâce à un paramètre nullable.

5. Compétences mobilisées

5.1. Compétences techniques

- Conception d'une base de données relationnelle à partir d'un MCD
- Génération automatique d'un script SQL via Looping
- Programmation orientée objet en C# (classes, héritage, encapsulation)
- Mise en œuvre du pattern Singleton pour la gestion de la connexion à la BDD
- Conception en architecture MVC
- Manipulation des données via ADO.NET et MySql.Data
- Création d'interfaces utilisateur avec Windows Forms
- Hashage de mot de passe avec SHA2-256
- Gestion des contraintes d'intégrité référentielle
- Génération de documentation technique avec SandCastle
- Création d'un installateur MSI

5.2. Compétences méthodologiques

- Lecture et compréhension d'un dossier documentaire (cas d'utilisation)
- Utilisation d'un gestionnaire de versions (Git) et d'un dépôt distant (GitHub)
- Suivi de tâches via un tableau Kanban
- Rédaction de commits explicites suivant la convention Conventional Commits
- Rédaction de documentation utilisateur (Readme + vidéo)

6. Conclusion

Cet atelier de professionnalisation m'a permis de mettre en pratique l'ensemble des compétences acquises au cours du Bloc 2 du BTS SIO : conception de bases de données, programmation orientée objet, architecture MVC, gestion de versions, documentation.

Le projet MediaTek86 est livré conforme au cahier des charges : tous les cas d'utilisation sont implémentés, la sécurité de l'accès à l'application et à la base de données est assurée, l'installateur permet un déploiement simple sur un poste utilisateur, et l'ensemble du code est documenté.

L'expérience acquise sur ce projet, notamment la rigueur du respect de l'architecture MVC, la traçabilité des modifications via Git, et la production d'une documentation complète, constitue un atout important pour ma future période de stage en entreprise.

— Fin du compte rendu —